

CHILLISOURCE GAME ENGINE PARTICLE STUDY

By

Angela Nicole Gross

Bachelor of Science, The University of Montana, Missoula, MT, 2014

Thesis

presented in partial fulfillment of the requirements
for the degree of

Master of Science
in Computer Science

The University of Montana
Missoula, MT

Autumn 2016

Approved by:

J.B. Alexander S. Ross Ph.D., Dean
Graduate School

Travis Wheeler Ph.D., Chair
Computer Science

Michael Cassens M.S.
Computer Science

Johnathan Bardsley Ph.D.
Mathematical Sciences

© COPYRIGHT

by

Angela Nicole Gross

2016

All Rights Reserved

Gross, Angela N., M.S., December 2016

Computer Science

ChilliSource Game Engine Particle Study

Chairperson: Travis Wheeler

This is the short summary of why this thesis is awesome and *YOU* really want to read it!

ACKNOWLEDGMENTS

Thanks guys ...

TABLE OF CONTENTS

COPYRIGHT	ii
ABSTRACT	iii
ACKNOWLEDGMENTS	iv
CODE LISTINGS	viii
LIST OF FIGURES	ix
LIST OF TABLES	x
CHAPTER 1 INTRODUCTION	1
1.1 Thesis Organization	1
CHAPTER 2 BACKGROUND	2
2.1 Particle Systems	2
2.1.1 Overview	2
2.1.2 Particle Model	3
2.1.2.1 Life Cycle	3
2.1.2.2 Particle Attributes	4
2.1.2.3 Hierarchy	5
2.2 The ChilliSource Game Engine	5
2.2.1 Overview	5
2.2.2 Architecture	6
2.2.3 Life Cycle	7

2.3	ChilliSource Particle Effect Components	10
2.3.1	Overview	10
2.3.2	Architecture	11
2.3.2.1	Overview	11
2.3.2.2	Particle Effect	11
2.3.2.3	Particle Drawable	14
2.3.2.4	Particle Emitter	14
2.3.2.5	Particle Affector	15
2.3.2.6	Particle Array	16
2.3.2.7	Concurrent Particle Data	16
2.3.2.8	Other Important Attributes	17
2.3.3	Life Cycle	18
2.3.3.1	Overview	18
2.3.3.2	Initialization Phase	18
2.3.3.3	Update Phase	18
2.3.3.4	Render Phase	21
2.3.3.5	Clean Up Phase	22
2.3.4	Usage	23
CHAPTER 3 THE CHILLISOURCE AUTOMATED PONG GAME . .		26
3.1	Overview	26
3.2	Architecture	26
3.3	Gameplay	26
CHAPTER 4 INSTRUMENTATION		27
4.1	Overview	27
4.2	Metrics System	27
4.3	Timing System	27

4.4	Output	27
4.5	Initial Results	27
CHAPTER 5 CHILLISOURCE PARTICLE EFFECT COMPONENT		
	CASE STUDIES	28
5.1	Overview	28
5.2	Using More Than One Mutex	28
5.2.1	Code	28
5.2.2	Discussion	28
5.2.3	Results	28
5.3	Using A Lock Free Data Structure	28
5.3.1	Code	28
5.3.2	Discussion	28
5.3.3	Results	28
CHAPTER 6 CHILLISOURCE PARTICLE EFFECT COMPONENT		
	OBSERVATIONS	29
6.1	Using Copies to Reduce Contention	30
6.1.1	Overview	30
6.1.2	Using the Concurrent Particle Data Object	30
6.1.3	Not Using the Concurrent Particle Data Object	30
6.1.4	Comparison	30
6.2	Multithreading and Task Scheduling	30
6.2.1	Overview	30
6.2.2	The ChilliSource Task Scheduler	30
6.2.3	Using the Small Task	30
6.2.4	Using the GameLogic Task	30
6.2.5	Using No Background Task	30

6.2.6	Comparison	30
6.3	Taxing the Engine and Emission Failure	30
6.3.1	Overview	30
6.3.2	Emission Failure	30
6.3.3	Emission Failure Source	30
6.3.4	Emission Failure Solution	30
BIBLIOGRAPHY		31

CODE LISTINGS

1	An example ChilliSource particle definition file for a generic particle effect. Note how the initial and changing-over-time (i.e. "affector") <i>Properties</i> can either be random values or constants.	12
2	The struct that is used to describe the properties of a particle within the <i>Particle Array</i> in <i>Particle Effect Component</i>	16
3	The struct that is used to describe the properties of a particle within the <i>Particle Array</i> in <i>Concurrent Particle Data</i>	17
4	The struct that is used to bundle all information needed by the background task performing the particle update.	19
5	Pseudocode describing how a <i>Particle Effect Component</i> is updated by scheduling a background task.	19
6	Pseudocode describing what CommitParticleData does when it copies its data to the <i>Concurrent Particle Data</i> instance.	21
7	Pseudocode showing how a <i>Particle Effect Component</i> uses its <i>Drawable</i> to render itself.	22
8	Creating a scene that adds a basic <i>Particle Effect Component</i> in ChilliSource.	23
9	The function to create an ambient light with the <i>Basic Entity Factory</i> state system. . .	24
10	Creating a basic particle effect component with ChilliSource.	24

LIST OF FIGURES

Figure 2.1	Particle effect created by William T. Reeves	2
Figure 2.2	Cubes emitting particles	3
Figure 2.3	Basic structure of a ChilliSource application	7
Figure 2.4	An example of the Update life cycle event	9
Figure 2.5	Basic structure of a ChilliSource Particle Effect Component	11

LIST OF TABLES

CHAPTER 1 INTRODUCTION

1.1 Thesis Organization

The rest of this thesis is organized as follows:

- **Chapter 2** Review of particle systems and the ChilliSource game engine
- **Chapter 3** Overview of the automated pong game that was created to facilitate case studies
- **Chapter 5** Discussion of the two case studies performed to explore contention in large data structures
- **Chapter 6** Discussion of three interesting aspects of the ChilliSource particle effect component

CHAPTER 2 BACKGROUND

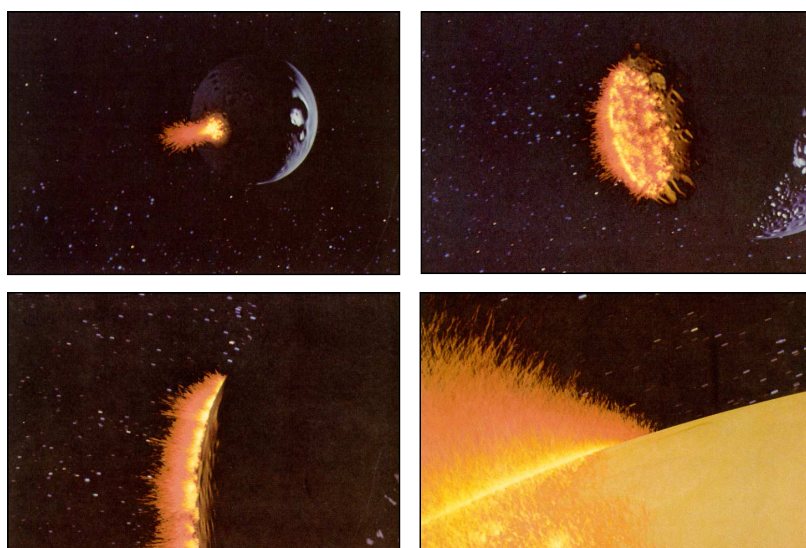


Figure 2.1: The particle effect created by William T. Reeves from the film *Star Trek II: The Wrath of Khan*

2.1 Particle Systems

2.1.1 Overview

Visual representation of "fuzzy objects", or objects that do not have well-defined surfaces, in computer graphics is generally considered a difficult task due to the non-deterministic changes in the fuzzy object's shape and form. When working on the film *Star Trek II: The Wrath of Khan*, a researcher at Lucasfilm Ltd., William T. Reeves, developed what he called a **particle system**, which is "... a collection of many minute particles that together represent a fuzzy object. Over a period of time, particles are generated into a system, move

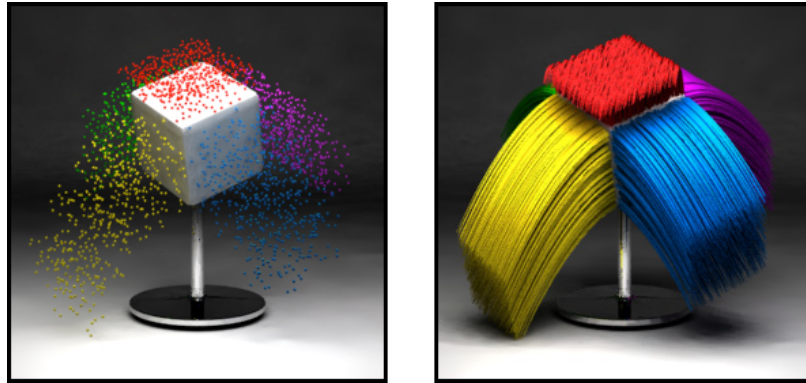


Figure 2.2: Left: A cube emitting a stream of 1000 animated particles from 5 of its 6 faces. Right: A cube emitting a stream of 1000 static particles, or strands, from 5 of its 6 faces

and change from within the system, and die from the system.” [1] Although the concept of modeling these objects as a collection of particles was not novel, Reeves' basic framework for particle systems is still widely used today to create complex visual effects in video games and films.

Fuzzy objects like sparks, clouds, stars, water, and crowds of people can be easily translated and modeled as a particle system, but particle systems can also be used to model objects that contain numerous strands like hair and grass. These objects can be represented by rendering every strand as the entire lifetime of a particle in the system and is then treated as a single strand. The particles in a system that have a distributed lifetime (e.g. sparks) are thought of as *animated* and particles that are rendered all at once (e.g. hair) are thought of as *static*. For the duration of our own study, we have exclusively focused on animated particles.

2.1.2 Particle Model

2.1.2.1 Life Cycle

A particle system has 4 main events that defines its life cycle:

1. Emission (or Generation, according to Reeves)

2. Update
3. Render
4. Death (or Extinction, according to Reeves)

The **Emission** event is the initial process in which particles are generated with their initial values. Often a "generation shape"[1] is used during the emission phase that defines the two or three dimensional space that the particles can originate, or emit, from. Emission can be increased or decreased in frequency using the *emission rate*, and the number of particles that enter the particle system each emission can also be changed using the *particles per emission* variable. After emission, particles in the particle system undergo the **Update** and **Render** events. These events are repeated in perpetuum until the particle dies, or undergoes the **Extinction** phase. The **Update** phase involves iterating over all particles throughout the system and updating their attributes (e.g. position, velocity, etc.). The **Render** phase, on the other hand, is the phase in which all of the particles are actually drawn to the screen.

2.1.2.2 Particle Attributes

As mentioned above, each individual particle in a particle system will have attributes that change over time. Although every particle system is different, the following is what Reeves considered to be the core attributes of a particle:

- Position
- Velocity
- Size
- Color
- Transparency
- Shape
- Lifetime

A particle has a position in two or three dimensional space, and its initial position is determined by the emission process. Other attributes that may also change over time

include its speed, direction, size, color, and transparency. Its lifetime, however, is pre-determined at emission, and it is the attribute that dictates how long the particle "lives", or remains on the screen. In order to decrease uniformity and increase realism, all of these values can vary across the particles in the system. For example, the particles in a given particle system could have a varying lifetime of 5 to 10 seconds so that the particles that shared an emission would not all die simultaneously.

2.1.2.3 Hierarchy

It is often convenient to design a particle system such that it is made up of a hierarchy of child particle systems. This hierarchy "can be used to exert global control on a complicated fuzzy object that is composed of many particle systems." [1] In other words, a fuzzy object could easily be transformed in overall shape, size, color, etc. by sweeping through the descendant particle systems. It also allows the grouping together of particle systems that have different types of particles. For example, a campfire fuzzy object would need smoke, sparks, and the flame itself. All of these components would have distinct particle definitions and would be separate particle systems. A particle system hierarchy, in this case, would allow the smoke, sparks, and flame particle systems to be bundled together as a single fuzzy object.

2.2 The ChilliSource Game Engine

2.2.1 Overview

Chosen for its open source and well-thought out architecture, ChilliSource is the game engine that was used in our study of particle systems. ChilliSource is a free, open source game engine created by ChilliWorks which is a group that is part of United Kingdom's Tag Games. It allows developers to create 2D and 3D content for iOS, Android, Kindle, and Windows operating systems in C++ 11. Some of its key features include networking, shader support, responsive (i.e. adaptive to different resolutions) graphical user interfaces, modular

and extensible lighting and shadows, skinned animation, Facebook connect integration, consistent API, and automatic builds of iOS/Android/Kindle applications. [2]

2.2.2 Architecture

The structure of a ChilliSource application is primarily composition based in that it relies on a hierarchy of modules to define its functionality. The main types of modules that comprise the application are *States*, *Systems*, *Entities* and *Components*. Figure 2.3 demonstrates the architecture of a ChilliSource application using these basic modules.

The *Application* instance initializes and manages *Application Systems*, which are modules that describe functionality used across the entire application. A non-exhaustive list of some application systems include the *Task Manager*, *Texture Provider*, *Render System*, and the *State Manager*. The *State Manager*, as shown in Figure 2.3, manages a stack of *States* and ensures that the state on top of the stack is active.

States can be thought of as different parts of the application such as a main menu, inventory, settings menu, pause screen, and the game scene itself. *States*, like the *Application*, initialize and manage *State Systems*. *State Systems* are similar to *Application Systems* except they describe functionality that only pertains to a single state and its lifetime is limited to that state. An important state system, the *Scene*, manage *Entities* that form that scene. The base *State* also creates and manages the *Canvas*, *Gesture*, and "User" state systems. The *Canvas* renders the user interface, the *Gesture* state system helps receive input events like pinching or tapping, and "User" state systems are custom modules built by the developer to describe functionality for the state.

Entities are essentially objects within the game that are defined by both (a) a transform describing their rotation, size, and position and (b) *Components*, which are modules that describe the entity's functionality. *Entities* can also contain a hierarchy of entities in which, much like in particle systems, their transforms and life cycle events are linked by the hierarchy. Some examples of *Components* are AI Controllers, Health, Damage, and Physics.

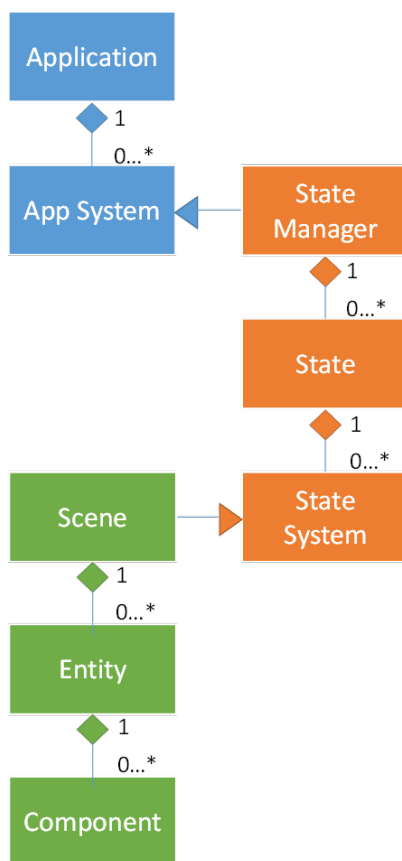


Figure 2.3: The basic structure of a ChilliSource application

ChilliSource's architecture favors composition over inheritance. For example, if we wanted to create enemy and player *Entities*, we may be tempted to create a base Human *Entity* class that had health related methods and attributes. In order to truly adhere to the spirit of ChilliSource's design, however, you would create a *Component* called something like **HealthComponent** that could be attached to any single *Entity*. In this way, ChilliSource lends itself to a great deal of code reusability and flexibility.

2.2.3 Life Cycle

The majority of game engines follow a life cycle very similar to the one for particle systems that Reeves' developed, which is:

1. Initialize
2. Update
3. Draw
4. Stop

The **Initialize** step usually involves loading textures, sounds, and anything else that the engine needs to do before starting the main game loop. The **Update** step is the heart of game, and is usually driven by some kind of state machine. Some examples of typical states include Menu, Game, Settings, and Exit. The **Draw** step is also part of the main game loop and, just as it sounds, this is where everything is drawn to the screen. Typically, there is some kind of renderer that handles all of the low-level calls to the graphics card (e.g. OpenGL, DirectX, etc.). And, optionally, there is the **Stop** step in which the game has been stopped and all assets and resources will be released. This may or may not be included as a separate state in the **Update** phase, but it is good to logically separate the two events.

ChilliSource, as you may expect, adheres to the basic "Initialize, Update, Draw, and Stop" phases. However, there are a great deal of other events that are perpetuated from the *Application* instance. The following is a list of the main life cycle events that all modules receive:

1. Init
 - This initializes the application and begins the **Update** event.
2. Update
 - This updates the application, its systems, and the state manager's states.
3. Render
 - This renders objects and user interface elements to the screen and is the main draw loop of the application.
4. Suspend

- This will notify active state(s) to pause and it will suspend all systems in reverse order.
5. Resume
 - This resumes the application from a suspended state.
 6. Background
 - This occurs when the application is not in view any longer.
 7. Foreground
 - This occurs when the application is in view again.
 8. Destroy
 - This cleans up the application and releases all systems in reverse order.

All of these events are kicked off from the *Application* instance. The flow of events looks something like *Application* → *Application Systems (State Manager)* → *States* → *State Systems (Scene)* → *Entities* → *Components*. Figure 2.4 shows what this flow looks like for the **Update** life cycle event using pseudocode. Note, however, that each event means something slightly different depending on which module it is called in (e.g. **Update**

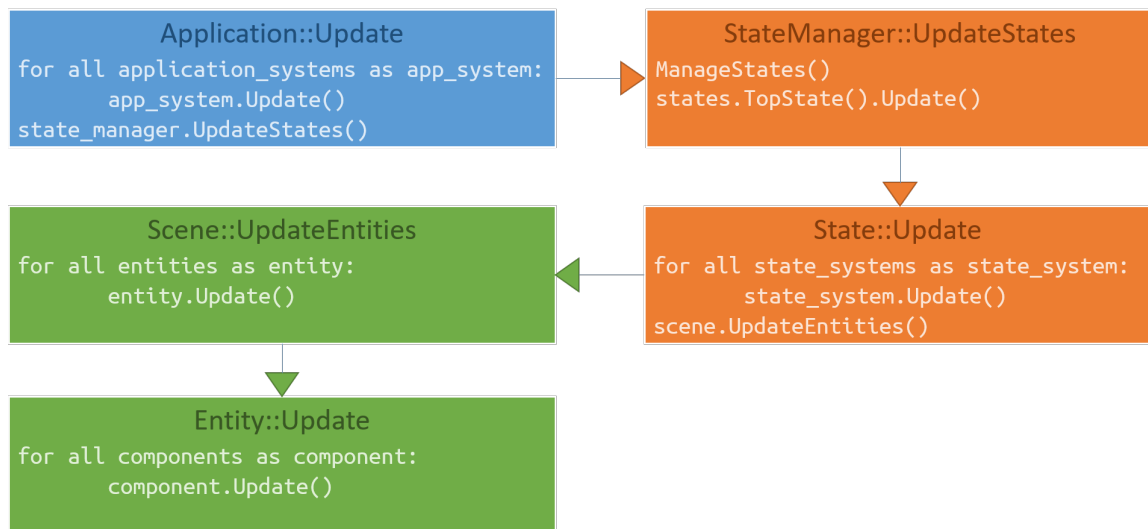


Figure 2.4: An example of the **Update** life cycle event being perpetuated from the *Application* all the way down to a *Component*

in the *Application* means something different than the **Update** in a *State*). Additionally, there are a number of other events that are not discussed here that only have meaning for specific modules (e.g. **OnAddedToEntity** for the *Component* module). See [3] for more information.

Although these are the main events that are automatically pushed by the application, the game developer can create custom events that can be subscribed to by entities, components, and systems. An *Event* can open, close, and notify *Connections*, where a *Connection* allows its user to listen for *Events*. Some examples of custom events are entity collisions, player death, cut scene trigger, game lost, game won, etc.

2.3 ChilliSource Particle Effect Components

2.3.1 Overview

A special type of component in ChilliSource is the *Particle Effect Component*. It inherits from the *Render Component* class¹, which is, just as it sounds, a component that is rendered to the screen. *Render Components* have methods for visibility, culling, casting shadows, and usage of its transformation matrix and sort predicates. They also have virtual methods that child classes can define for materials, transparency, rendering, and bounding boxes. For the purpose of our study, however, the exact details of how a *Render Component* works is outside of our scope. It is only necessary to know that the *Particle Effect Component* is not only updated by its parent entity but is also rendered by the *Application*'s renderer.

¹It's important to note here that the version of ChilliSource that we are using is 1.6.0 and, at the time of writing this paper, it is currently at version 2.0.0. In version 2.0.0, the rendering system is completely overhauled and *Particle Effect Components* are *Volume Components* and not *Render Components*. This new system was not studied and will not be discussed in depth here.

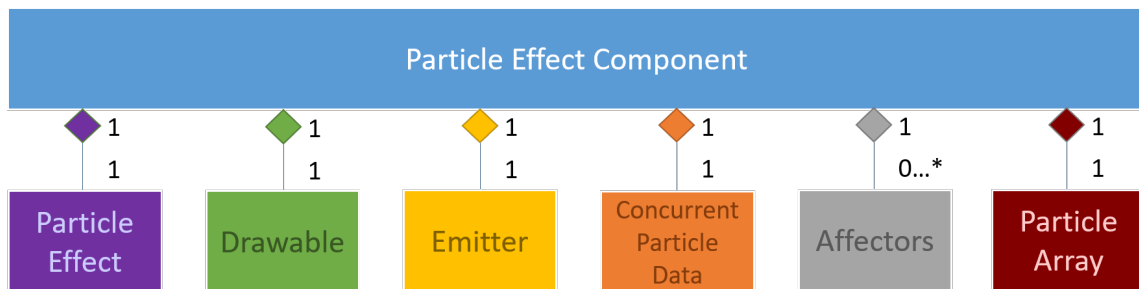


Figure 2.5: The basic structure of a ChilliSource Particle Effect Component

2.3.2 Architecture

2.3.2.1 Overview

The structure of a *Particle Effect Component* in ChilliSource is fairly complex. Its composition includes a number of objects, primitives, and data structures, but we can boil it down to 6 parts:

- A *Particle Effect* object
- A *Particle Drawable* object
- An *Particle Emitter* object
- A list of *Particle Affectors* (optional)
- A *Concurrent Particle Data* object
- An array of *Particle* structs

2.3.2.2 Particle Effect

The *Particle Effect* object contains all of the *Properties* and *Definitions* for the effect. A *Property* defines a value that may be more complex than just a single value. For example, a *Property* could be defined as a random value selected within a certain range, it may be a value that changes over the lifetime of the particle effect, or it could simply be a constant primitive value. *Definitions*, on the other hand, describe a number of properties that should be used to create an object. The *Properties* and *Definitions* that the *Particle Effect* contains

include:

- The particle effect's duration
- The total maximum number of particles used in the effect
- The space in which the particle effect is simulated (either "local" or "world")
- The lifetime property of a new particle generated in the effect
- The initial scale property of a new particle generated in the effect
- The initial rotation property of a new particle generated in the effect
- The initial color property of a new particle generated in the effect
- The initial speed property of a new particle generated in the effect
- The initial angular velocity property of a new particle generated in the effect
- The particle effect's drawable definition
- The particle effect's emitter definition
- The particle effect's affector definitions

These *Properties* and *Definitions* are used to create instances and values that will be used by the *Particle Effect Component*. Thus, the contents of the *Particle Effect* instance completely determines the behavior and appearance of the *Particle Effect Component*. In order to create these intricate *Particle Effects*, we must utilize a JSON particle definition file like the one shown in Code Listing 1. Given the length and verbosity of this file, it is easy to see that defining a particle effect by hand in C++ would quickly become messy and difficult to maintain.

Code Listing 1: An example ChilliSource particle definition file for a generic particle effect. Note how the initial and changing-over-time (i.e. "affector") *Properties* can either be random values or constants.

```

1  {
2      "Duration": "1.0",
3      "MaxParticles": "10000",
4      "SimulationSpace": "World",
5      "LifetimeProperty": {
6          "Type": "RandomConstant",
7          "LowerValue": "0.8",

```

```

8     "UpperValue": "1.2"
9 },
10 "InitialScaleProperty": {
11     "Type": "RandomConstant",
12     "LowerValue": "0.5 0.5",
13     "UpperValue": "1.2 1.2"
14 },
15 "InitialRotationProperty": "0.0",
16 "InitialColourProperty": "0.0 0.0 0.0 0.0",
17 "InitialSpeedProperty": "0.5",
18 "InitialAngularVelocityProperty": "0.0",
19 "Drawable": {
20     "Type": "StaticBillboard",
21     "MaterialPath": "Particle/Particle.csmaterial",
22     "AtlasPath": "TextureAtlases/Particle/Particle.csatlas",
23     "ImageIds": "Generic",
24     "Size": "1.0 1.0",
25     "SizePolicy": "FitMaintainingAspect"
26 },
27 "Emitter": {
28     "Type": "Cone",
29     "EmissionMode": "Stream",
30     "EmitFromType": "Inside",
31     "EmitDirectionType": "AwayFromBase",
32     "EmissionRateProperty": "100.0f",
33     "ParticlesPerEmissionProperty": "10000",
34     "EmissionChanceProperty": "1.0",
35     "RadiusProperty": "0.5",
36     "AngleProperty": "0.8"
37 }
38 "Affectors": [
39     {
40         "Type": "ColourOverLifetime",
41         "TargetColourProperty": "0.0 0.0 0.0 0.0",
42         "IntermediateColours": [
43             {
44                 "ColourProperty": {
45                     "Type": "RandomConstant",
46                     "LowerValue": "0.9 0.4 0.12 0.0",
47                     "UpperValue": "1.0 0.25 0.0 0.5"
48                 },
49                 "TimeProperty": "0.15"
50             },
51             {
52                 "ColourProperty": {
53                     "Type": "RandomConstant",
54                     "LowerValue": "0.9 0.4 0.12 0.0",
55                     "UpperValue": "1.0 0.25 0.0 0.5"
56                 },
57                 "TimeProperty": "0.75"
58             }
59         ]
60     },
61     {
62         "Type": "ScaleOverLifetime",

```

```

63     "ScaleProperty": {
64         "Type": "RandomConstant",
65         "LowerValue": "0.75 1.5",
66         "UpperValue": "1.0 2.0"
67     }
68 }
69 ]
70 }

```

2.3.2.3 Particle Drawable

The *Drawable* object is responsible for rendering all of the particles that belong to its *Particle Effect Component*. A *Static Billboard Particle Drawable*², which is a *Drawable* that renders each and every individual particle as camera facing sprites, is used to accomplish this. The *Drawable* will always render particles on the main game thread.

2.3.2.4 Particle Emitter

The *Emitter* object is used to spawn new particles during **Emission**. The *Emitter* gets to decide the direction, location, and moment that a new particle emits. Unlike the rendering that a *Drawable* does, the *Emitter* emits particles as a background task. There are 5 different kinds of *Emitters*, and their differences lie in the "generation", or emission, shape:

- *Point Particle Emitter*
 - This emitter spawns particles at the parent entity's position with a random direction.
- *Circle Particle Emitter*
 - This emitter spawns particles either within a circle or on the circle's perimeter.

It can be set to emit particles with a random direction or a direction away from

²As of version 1.6.0, the only *Drawable* type available to us is the *Static Billboard*, but the current architecture suggests that there could be different types of *Drawables* in future versions.

the center of the circle.

- *Sphere Particle Emitter*
 - Similar to the *Circle Particle Emitter*, this emitter spawns particles either within a sphere or on the sphere’s surface. It can be set to emit particles with a random direction or a direction away from the center of the sphere.
- *Cone Particle Emitter*
 - This emitter spawns particles either within a cone or on the cone’s surface. It can be set to emit particles with a random direction or a direction away from the base of the cone.
- *Cone 2D Particle Emitter*
 - This emitter spawns particles either within a 2D cone, on the 2D cone’s edges, or at the 2D cone’s base. It can be set to emit particles with a random direction or a direction away from the base of the 2D cone.

2.3.2.5 Particle Affector

A *Particle Affector* object applies an effect to all particles that change over time. A *Particle Effect Component* can utilize zero or more *Affectors*, and, like the *Emitter*, *Affectors* update particles in a background thread. There are 4 different kinds of *Affectors* that can be applied:

- *Scale Over Lifetime Particle Affector*
 - This affector changes the size of the particles over their lifetime by using a calculated scale factor. This factor will be used to help interpolate the particle’s size across updates.
- *Color Over Lifetime Particle Affector*
 - This affector changes the color of particles over their lifetime by using its initial and target colors to generate interpolated colors across updates.
- *Acceleration Lifetime Particle Affector*

- This affector changes the speed of particles over their lifetime.
- *Angular Acceleration Lifetime Particle Affector*
 - This affector changes the angular velocity of particles over their lifetime.

2.3.2.6 Particle Array

The *Particle Array* is a data structure that holds all of the particles that comprise the *Particle Effect Component*, and it is used by the *Drawable*, *Emitter*, *Affector*, and *Concurrent Particle Data* instances. Each particle is represented by a *Particle* struct, as shown in Code Listing 2. We can observe that the information provided by this struct mirrors the particle attributes that Reeves used in his particle systems.

Code Listing 2: The struct that is used to describe the properties of a particle within the *Particle Array* in *Particle Effect Component*.

```

1  struct Particle final
2  {
3      bool m_isActive = false;
4      f32 m_lifetime = 0.0f;
5      f32 m_energy = 0.0f;
6      Vector3 m_position;
7      Vector2 m_scale = Vector2::k_one;
8      f32 m_rotation = 0.0f;
9      Colour m_colour = Colour::k_white;
10     Vector3 m_velocity;
11     f32 m_angularVelocity = 0.0f;
12 };

```

2.3.2.7 Concurrent Particle Data

The *Concurrent Particle Data* object is a container for data that is shared by the background and main threads. *Concurrent Particle Data* is written to by the same background thread that emits and affects particles, and it is read by the *Particle Effect Component's Drawable* in the main thread. It contains, essentially, copies of important attributes that are shared by these threads. This includes an array of particles, indices of particles that

were just emitted, and the bounds of the particle effect. Thus, all thread synchronization and locking occurs in *Concurrent Particle Data* and not in the *Particle Effect Component* itself.

It is important to note that *Concurrent Particle Data*'s array of particles uses a different struct than the one used in *Particle Effect Component*. As we can see in Code Listing 3, the struct contains a subset of the information that is provided by *Particle Effect Component*'s *Particle* struct. This is intentional, as we only want to copy the information that will be used by the *Drawable* later.

Code Listing 3: The struct that is used to describe the properties of a particle within the *Particle Array* in *Concurrent Particle Data*.

```

1  struct Particle final
2  {
3      bool m_isActive = false;
4      Vector3 m_position;
5      Vector2 m_scale = Vector2::k_zero;
6      f32 m_rotation = 0.0f;
7      Colour m_colour = Colour::k_white;
8  };

```

2.3.2.8 Other Important Attributes

There are a number of other important attributes that are not the 5 parts discussed above. These include:

- Bounding Box Objects
- Playback Enumerators and Floats
- Custom Events

The *Bounding Box Objects* are containers that describe the effect's shape and are used to aide in the rendering of the effect. The *Playback Enumerators*, *Playback Type* and *Playback State*, help describe the whether or not the particle effect loops and the current life cycle phase of the particle effect, respectively. The *Playback Floats* keep track of how long the

particle effect has been playing. The two *Custom Events* that the effect uses are fired off when the particle effect fully completes (i.e. no more particles are left alive) and when a particle effect finishes emitting.

2.3.3 Life Cycle

2.3.3.1 Overview

The life cycle of a *Particle Effect Component* is very similar to the one that was developed by Reeves, and it can be described as 4 main phases:

- Initialization
- Update
- Render
- Clean Up

2.3.3.2 Initialization Phase

During the **Initialization** phase, the *Particle Effect Component* creates the *Drawable*, *Emitter*, *Concurrent Particle Data*, and, if specified by the *Particle Effect* instance, its *Affectors*. It also creates the *Particle Array* and initializes the *Bounding Shapes*.

2.3.3.3 Update Phase

The **Update** phase uses 4 states to characterize the current playback state of the particle effect, which are **Starting**, **Playing**, **Stopping**, and **Not Playing**. During the **Starting** state, particles are initialized to be inactive before it updates the particle effect for the first time and enters the **Playing** state. The **Playing** state is when particles are rendered and updated. If, during the **Playing** state, the particle effect may enter a **Stopping** state if the playback timer exceeds the effect's duration. However, it may go back to a **Playing** state if active particles still remain within the *Particle Array*. When the particle effect's duration time has elapsed and there are no more active particles, the particle effect finally enters the

Not Playing state and the particle effect no longer does anything. The bulk of a particle effect's time is spent during the **Playing** state, and that is the state of the **Update** phase that we will be focusing on throughout our study and will be discussing at length.

Code Listing 4: The struct that is used to bundle all information needed by the background task performing the particle update.

```

1  struct ParticleUpdateDesc final
2  {
3      ParticleEffectCSPtr m_particleEffect;
4      ParticleEmitterSPtr m_particleEmitter;
5      std::vector<ParticleAffectorSPtr> m_particleAffectors;
6      std::shared_ptr<dynamic_array<Particle>> m_particleArray;
7      ConcurrentParticleDataSPtr m_concurrentParticleData;
8      f32 m_playbackTime = 0.0f;
9      f32 m_deltaTime = 0.0f;
10     Vector3 m_entityPosition;
11     Vector3 m_entityScale;
12     Quaternion m_entityOrientation;
13     bool m_interpolateEmission = false;
14 };

```

When the particle effect is updating, it prepares a struct containing pertinent information to hand over to a background task. This struct contains pointers to the *Particle Effect*, *Emitter*, *Affector(s)*, *Particle Array*, *Concurrent Particle Data*. It additionally holds some other miscellaneous information, such as the effect's parent entity's position, scale, and orientation. The struct in its entirety is shown in Code Listing 4. After it prepares the struct, it uses the *Application Task Scheduler* application system to schedule a background task. This process is shown using pseudocode in lines 1 - 11 in Code Listing 5.

As mentioned before in Section 2.3.2, the **Particle Update Task** updates particles from the *Particle Array*, applies *Affectors* to the particles, emits particles using the *Emitter*, calculates its new *Bounding Shapes*, and it updates *Concurrent Particle Data* with the new information. Lines 13 - 41 demonstrate these actions in Code Listing 5.

Code Listing 5: Pseudocode describing how a *Particle Effect Component* is updated by scheduling a background task.

```

1 ParticleEffectComponent::OnUpdate()
2 {
3     // Get a struct containing copies (and pointers) to pass to the task
4     copiedAtts = CopyUpdateAttributes();
5
6     // Schedule a task to update the particle in the background
7     ApplicationTaskScheduler.ScheduleTask
8     ({
9         ParticleUpdateTask(copiedAtts);
10    });
11 }
12
13 ParticleUpdateTask(copiedAtts)
14 {
15     // Update particles if they are active
16     for(particle : copiedAtts.particleArray)
17     {
18         if(particle.isActive)
19         {
20             particle.UpdateValues();
21         }
22     }
23
24     // Apply affectors
25     for(affector : copiedAtts.particleAffectors)
26     {
27         affector.AffectParticles();
28     }
29
30     // Try to emit
31     newParticleIndicesArray = copiedAtts.particleEmitter.EmitParticles();
32
33     // Update bounding shapes
34     AABB, Sphere = CalculateBoundingShapes();
35
36     // Copy this updated data to the concurrent particle data object
37     copiedAtts.particleConcurrentParticleData.CommitParticleData
38         (particleArray,
39          newParticleIndicesArray,
40          AABB, Sphere);
41 }

```

Code Listing 6 illustrates how the function, **CommitParticleData**, copies data from the background task to the *Concurrent Particle Data* instance. The function first copies all of the particles in the *Particle Array*, and only a subset of information that is related to drawing particles is copied per particle. It then appends the indices of the particles that were just newly emitted to its member array, *newParticleIndices*. At the end, it copies the

Bounding Objects. During all of this, the *Concurrent Particle Data* instance is locked by locking a member variable mutex.

Code Listing 6: Pseudocode describing what **CommitParticleData** does when it copies its data to the *Concurrent Particle Data* instance.

```

1 ConcurrentParticleData::CommitParticleData (particleArray,
2                                           newParticleIndicesArray,
3                                           AABB, Sphere)
4 {
5     // Lock the mutex for the scope of the whole function
6     using lock (this.mutex)
7     {
8         // Copy over all particles to the member particle array
9         for (i = 0 to particleArray.size())
10            {
11                this.particleArray[i] = particleArray[i].copySubset();
12            }
13
14        // Append the new indices to the member new indices array
15        this.newParticleIndices.append( newParticleIndices.copy() );
16
17        // Copy the bounding information to the member variables
18        this.AABB = AABB.copy();
19        this.Sphere = Sphere.copy();
20    }
21 }

```

2.3.3.4 Render Phase

Unlike the **Update** phase, the **Render** phase is not directly called by the *Particle Effect Component*. Instead, it is called by the *Renderer* application system. The *Renderer* retrieves the active scene from the *State Manager*, filters entities added to the scene that have *Render Components*, and attempts to render them.

The **Render** function uses the *Drawable* instance to draw the *Particle Effect Component*. The *Drawable*, in turn, uses *Concurrent Particle Data* to read the information needed about the particle effect in order to render it. The **Draw** function within the *Drawable* first locks the *Concurrent Particle Data* instance using a **Lock** function, and then proceeds to work with the copied data in *Concurrent Particle Data*. It uses the *newParticleIndices* to register

newly emitted particles to the *Static Billboard* that is used for rendering, and continues on to iterate over the copied array of particles in order to draw each particle that is currently active. When it is finished, it unlocks the *Concurrent Particle Data* instance.

Code Listing 7: Pseudocode showing how a *Particle Effect Component* uses its *Drawable* to render itself.

```

1 ParticleEffectComponent::Render(camera)
2 {
3     // Use the Particle Drawable to draw particles to the screen
4     if(this.playbackState == playing or this.playbackState == stopping)
5     {
6         this.drawable->Draw(camera);
7     }
8 }
9
10 Drawable::Draw(camera)
11 {
12     // Lock the Concurrent Particle Data object
13     this.concurrentParticleData->Lock();
14
15     // Activate the newly emitted particles in the static billboard
16     for(int index : this.concurrentParticleData->GetNewParticleIndices())
17     {
18         ActivateParticleInBillboard(index);
19     }
20
21     // Iterate through all of the particles and draw each one
22     DrawParticles(this.concurrentParticleData->GetParticleArray(), camera);
23
24     // Unlock the Concurrent particle Data object
25     this.concurrentParticleData->Unlock();
26 }

```

2.3.3.5 Clean Up Phase

Of all phases, this is one is by far the simplest. It releases all resources and clears allocated arrays. Specifically, it cleans up the *Particle Array*, the *Concurrent Particle Data* instance, the *Drawable* instance, the *Emitter* instance, and the array of *Affectors*.

2.3.4 Usage

Creating a basic scene that uses a *Particle Effect Component* in ChilliSource is fairly simple. As Code Listing 8 illustrates, it requires that we first create a camera along with a light source and add them both to the current *State's Scene*. Although a state system, *Basic Entity Factory*, abstracts some of this code, Code Listing 9 demonstrates that utilizing ChilliSource's API to create an ambient light is still easy to use and understand. We did not need to do this, but we also used the *Basic Entity Factory* to create a room in our scene.

Code Listing 8: Creating a scene that adds a basic *Particle Effect Component* in ChilliSource.

```

1  void State::OnInit()
2  {
3      // Retrieve the resource pool application system to create a particle effect
4      // Set the state's current scene's clear color to black
5      GetScene()->SetClearColour(CS::Colour::k_black);
6
7      // Retrieve a state system that will help create basic entities like
8      // cameras, rooms, and lights.
9      auto basicEntityFactory = CS::Application::Get()
10         ->GetSystem<Common::BasicEntityFactory>();
11
12     // Create a basic room and add it to the scene
13     CS::EntitySPtr room = basicEntityFactory->CreateRoom();
14     room->GetTransform().SetPosition(0.0f, 10.0f, 0.0f);
15     GetScene()->Add(room);
16
17     // Create a third person camera and add it to the scene
18     auto camera = basicEntityFactory
19         ->CreateThirdPersonCamera(room, CS::Vector3(0.0f, -9.0f, 0.0f));
20     GetScene()->Add(std::move(camera));
21
22     // Create an ambient light and add it to the scene
23     CS::EntitySPtr ambientLight = basicEntityFactory
24         ->CreateAmbientLight
25         (CS::Colour(0.65f, 0.65f, 0.65f, 1.0f));
26     GetScene()->Add(ambientLight);
27
28     // Create a particle effect entity and add it to the scene
29     auto particleEntity = CreateParticleEffect();
30     GetScene()->Add(std::move(particleEntity));
31 }

```

Code Listing 9: The function to create an ambient light with the *Basic Entity Factory* state system.

```

1 CS::EntityUPtr BasicEntityFactory::CreateAmbientLight
2                                     (const CS::Colour& in_colour)
3 {
4     // Use the application system, Render Component Factory, to create an
5     // ambient light.
6     auto ambientLightComponent = m_renderComponentFactory
7                                     ->CreateAmbientLightComponent(in_colour);
8
9     // Create an entity, and attach the ambient light component to it.
10    auto entity = CS::Entity::Create();
11    entity->AddComponent(ambientLightComponent);
12
13    // Return the entity
14    return entity;
15 }

```

Once we've added a room, lighting, and a camera to our scene, we create a particle effect entity using a function called **CreateParticleEffect**. The exact code inside of this function is shown in Code Listing 10. Inside of this function, we use the main *Application* instance to generate a *Particle Effect* from a csparticle file, and it accomplishes this by using a *Resource Pool* application system. Once we have a *Particle Effect*, we can use a *Render Component Factory* application system to create a *Particle Effect Component* using that *Particle Effect*. Lastly, we must add the *Particle Effect Component* to an entity and add that entity to the current state's scene.

Code Listing 10: Creating a basic particle effect component with ChilliSource.

```

1 CS::EntityUPtr State::CreateParticleEffect()
2 {
3     // Retrieve the singleton application instance through a static method
4     auto app = CS::Application::Get();
5
6     // Retrieve the resource pool application system to create a particle effect
7     // from a JSON csparticle file
8     auto resourcePool = app->GetResourcePool();
9     auto particleEffect = resourcePool
10         ->LoadResource<CS::ParticleEffect>
11         (CS::StorageLocation::k_package,
12          "Particle/generic.csparticle");

```

```

13
14 // Using a render component factory application system and the particle
15 // effect, create a particle effect component
16 auto renderComponentFactory = app->GetSystem<CS::RenderComponentFactory>();
17 auto particleComponent = renderComponentFactory
18     ->CreateParticleEffectComponent(particleEffect);
19
20 // Set the particle component's playback to looping
21 particleComponent
22     ->SetPlaybackType(CS::ParticleEffectComponent::PlaybackType::k_looping);
23
24 // Create an entity, and add the particle component to the entity
25 auto particleEntity = CS::Entity::Create();
26 particleEntity->AddComponent(std::move(particleComponent));
27
28 return particleEntity;
29 }

```

Compared to Reeves' particle systems, we can observe here that a *Particle Effect Component* is not composed of a hierarchy of child *Particle Effect Components*. As mentioned in Section 2.2.2, however, *Entities* are made up of *Components* and child *Entities*. Thus, ChilliSource's architecture allows the developer to create parent and child *Particle Effect Components* by nesting *Entities*.

CHAPTER 3 THE CHILLISOURCE AUTOMATED PONG GAME

3.1 Overview

3.2 Architecture

3.3 Gameplay

CHAPTER 4 INSTRUMENTATION

4.1 Overview

4.2 Metrics System

4.3 Timing System

4.4 Output

4.5 Initial Results

CHAPTER 5 CHILLISOURCE PARTICLE EFFECT COMPONENT CASE STUDIES

5.1 Overview

5.2 Using More Than One Mutex

5.2.1 Code

5.2.2 Discussion

5.2.3 Results

5.3 Using A Lock Free Data Structure

5.3.1 Code

5.3.2 Discussion

5.3.3 Results

CHAPTER 6 CHILLISOURCE PARTICLE EFFECT COMPONENT OBSERVATIONS

6.1 Using Copies to Reduce Contention

6.1.1 Overview

6.1.2 Using the Concurrent Particle Data Object

6.1.3 Not Using the Concurrent Particle Data Object

6.1.4 Comparison

6.2 Multithreading and Task Scheduling

6.2.1 Overview

6.2.2 The ChilliSource Task Scheduler

6.2.3 Using the Small Task

6.2.4 Using the GameLogic Task

6.2.5 Using No Background Task

6.2.6 Comparison

6.3 Taxing the Engine and Emission Failure

6.3.1 Overview

6.3.2 Emission Failure

6.3.3 Emission Failure Source

6.3.4 Emission Failure Solution

BIBLIOGRAPHY

- [1] W. T. Reeves, “Particle systems – a technique for modeling a class of fuzzy objects,” in *Proceedings of the 10th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’83. New York, NY, USA: ACM, 1983, pp. 359–375. [Online]. Available: <http://doi.acm.org/10.1145/800059.801167>
- [2] ChilliSource. (2014) Features. [Online]. Available: <http://www.chillisourceengine.com/features>
- [3] ——. (2014) Life cycle events. [Online]. Available: <http://www.chillisourceengine.com/life-cycle-events>
- [4] ——. (2014) Basic structure. [Online]. Available: <http://www.chillisourceengine.com/basic-structure>
- [5] Halixi72. (2007) Particle emitter. [Online]. Available: https://en.wikipedia.org/wiki/File:Particle_Emitter.jpg
- [6] ——. (2007) Strand emitter. [Online]. Available: https://en.wikipedia.org/wiki/File:Strand_Emitter.jpg
- [7] ChilliSource. (2016) Chillisource v1.6.0 code repository. [Online]. Available: <https://bitbucket.org/Chilli-Ian/chillisource-ag/src/781988bb9a8ee518c4ea6545ac0e5c55238ec562?at=temp/particleMetrics>